

로봇팔 기반의 생체영상 인식을 위한 딥러닝

2024 하계 실습생 김민오

2024-07-30

Contents



- 01 |
 - 실습계획수립
 - 개발환경구축
- 02 |
 - Object Detection
 - 데이터준비 및 라벨링
 - 모델학습 및 성능평가
 - 개선작업 및 최종결과
- 03 |
 - Instance Segmentation
 - 데이터준비 및 라벨링
 - 모델학습 및 성능평가
 - 개선작업 및 최종결과
- 04 |
 - Real-time Streaming
 - Object Detection
 - Instance Segmentation

01 실습계획수립

1 병증 발현단계 탐지

과수(사과나무)의 병충해 조기탐지하기 위한
딥러닝 모델 개발

Object Detection / Instance Segmentation

2 업무계획

컴퓨터세팅

데이터라벨링

모델학습

성능평가 및 개선

3 현장체험 및 최종발표

Real-time streaming 환경에서의 모델 시험
한 달간의 성과 발표



01 개발환경구축

1 개발도구

Pycharm 2024.1.4 (Professional Edition)

Jupyter Notebook 7.0.8

2 프로그래밍언어 및 패키지 관리

Python 3.8.0

Anaconda3 24.5.0

3 데이터라벨링 도구

LabelImg 1.8.6

LabelMe 5.5.0

4 모델학습 및 배포

Ultralytics 8.0.238

Object Detection : YOLOv8n / YOLOv8x / YOLOv8x6

Instance Segmentation : YOLOv8n-seg / YOLOv8x-seg



02 Object Detection

데이터준비 및 라벨링

1주차 (CPU)

50장의 RGB 이미지 라벨링 진행



02 Object Detection

모델학습 및 성능평가

1주차 (CPU)

YOLOv8n 20 epoch 학습 진행



문제 1 : SSL Error

```
requests.exceptions.SSLError: HTTPSConnectionPool(host='api.github.com', port=443): Max retries exceeded with url: /repos/ultralytics/assets/releases/tags/v8.2.0 (Caused by SSLError(SSLCertVerificationError(1, '[SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: self signed certificate in certificate chain (_ssl.c:1131)')))) (.venv) PS C:\Users\USER\PycharmProjects\KIMM_kimmo_Project\Day_3>
```

해결 : Library 버전 재통일 / 경로설정

02 Object Detection

모델학습 및 성능평가

문제 2 : None Detection



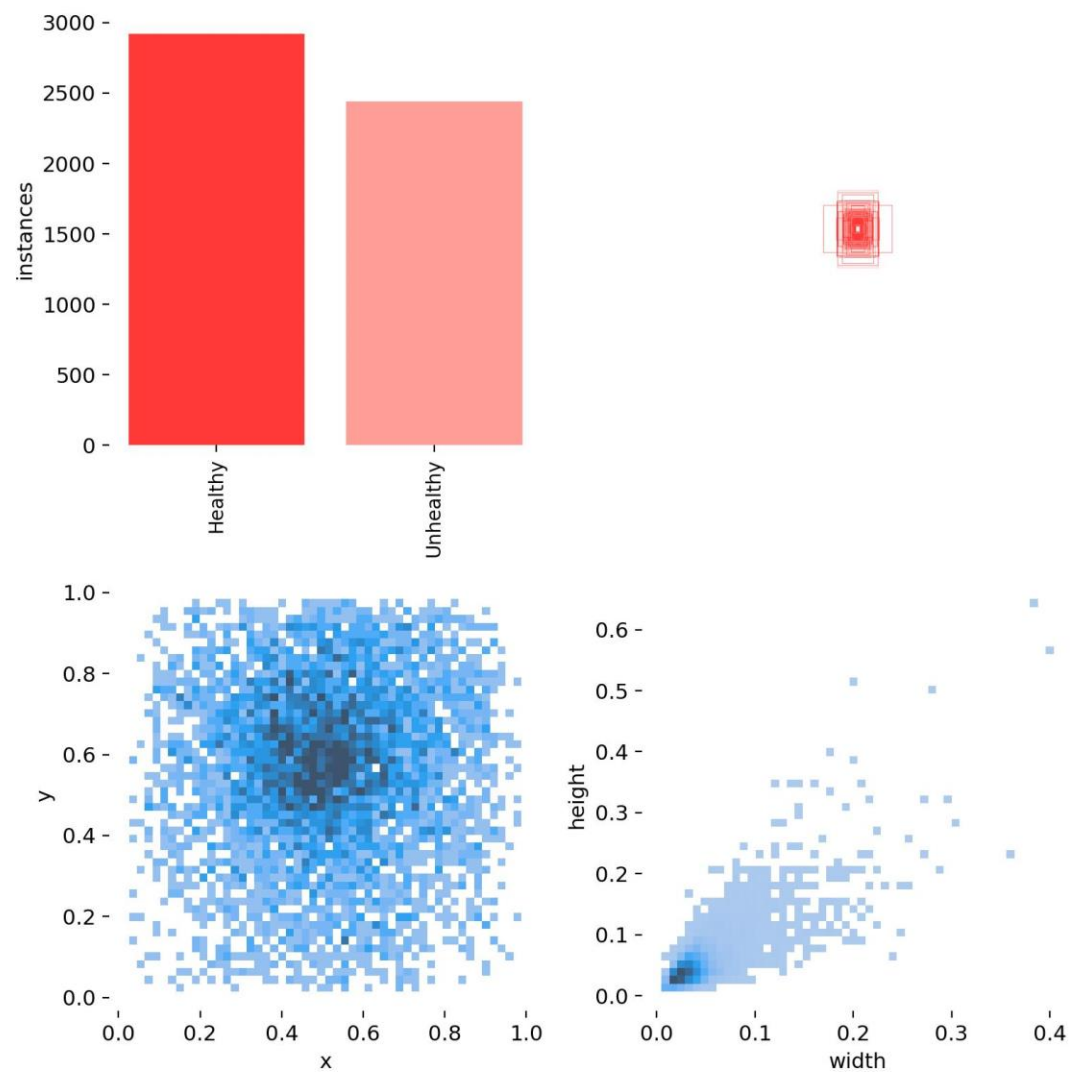
해결 : 데이터셋의 추가 및 epoch 증가 (CPU → GPU), 모델변경 (YOLOv8n → YOLOv8x)

02 Object Detection

개선작업 및 최종결과

2~3주차 (GPU)

증가된 데이터셋



모델 변경 (YOLOv8x → YOLOv8x6 (IMG Size 640 → 1280))

02 Object Detection

개선작업 및 최종결과

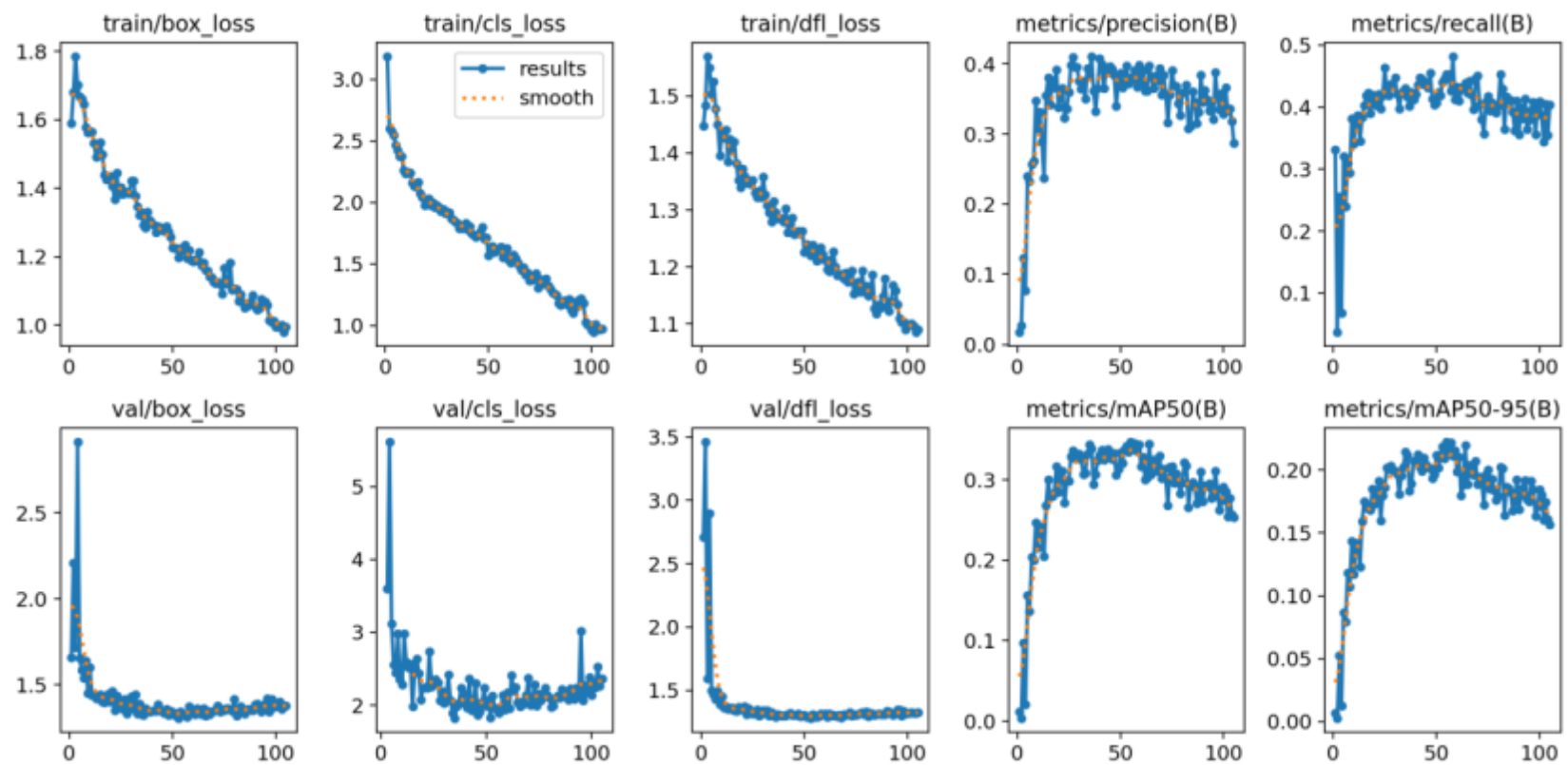
최종모델 Video Prediction (IMG 1280, 맵프레임마다)



02 Object Detection

개선작업 및 최종결과

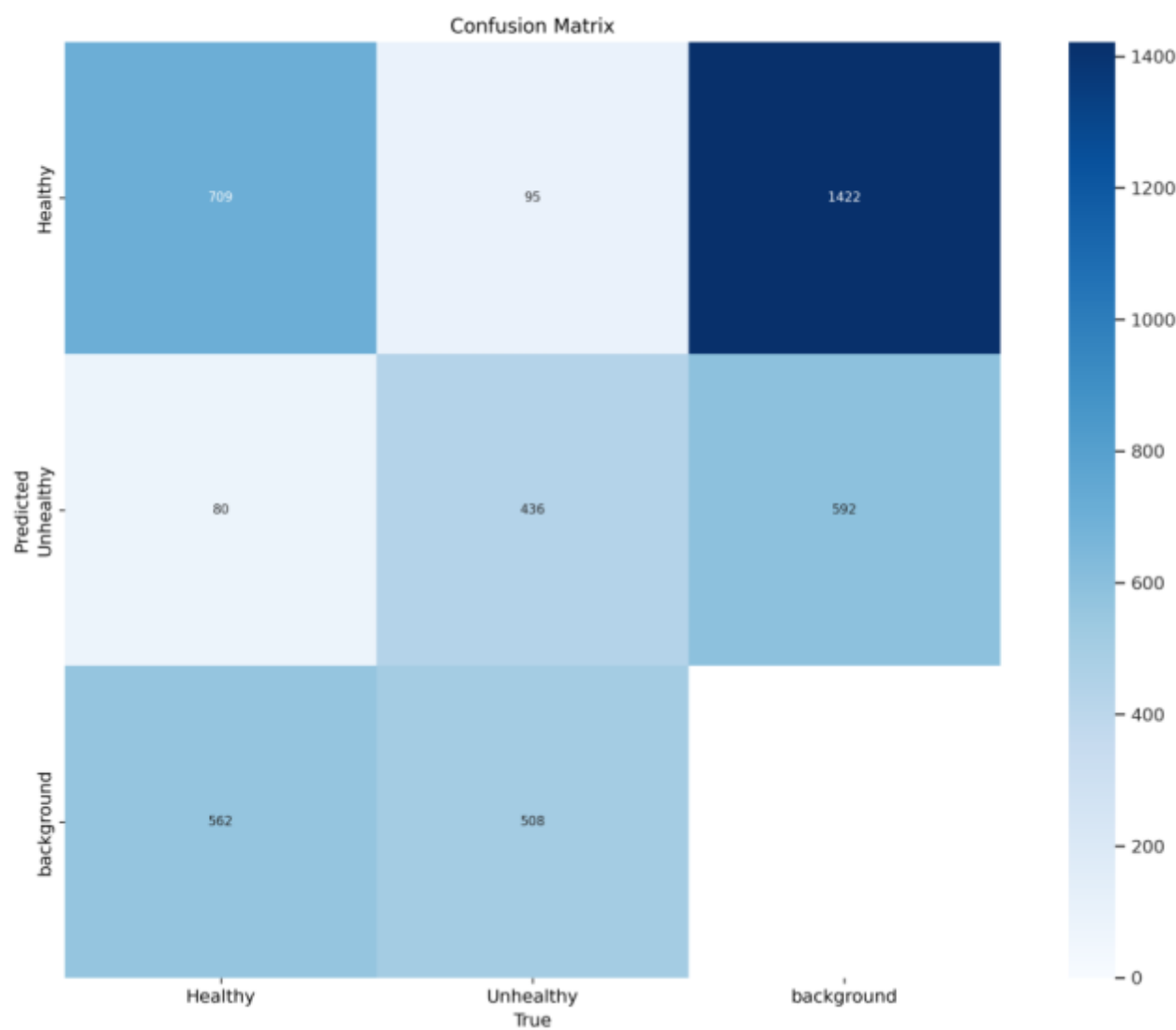
Performance - Result



02 Object Detection

개선작업 및 최종결과

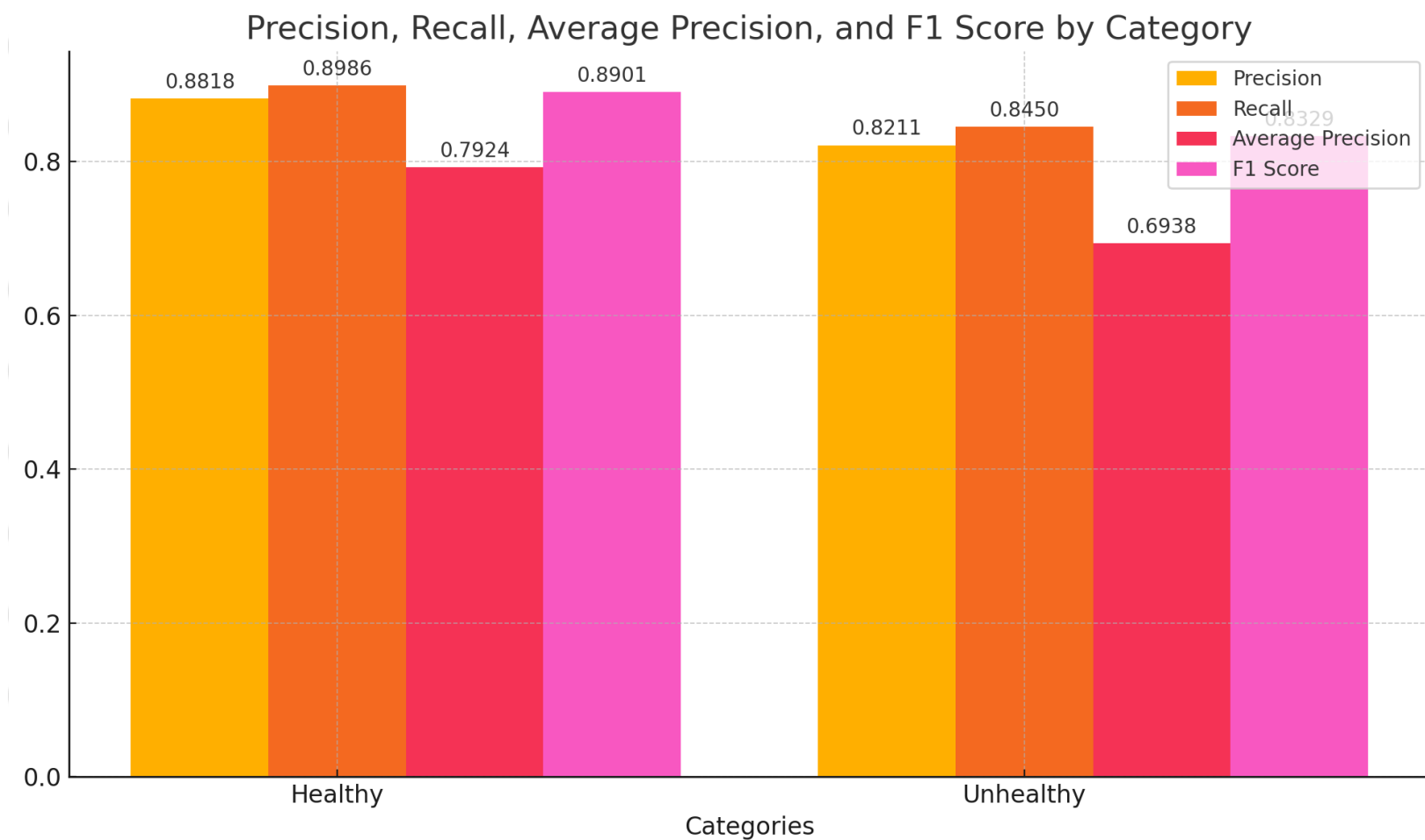
Performance – Confusion Matrix



02 Object Detection

개선작업 및 최종결과

Performance – Except Background



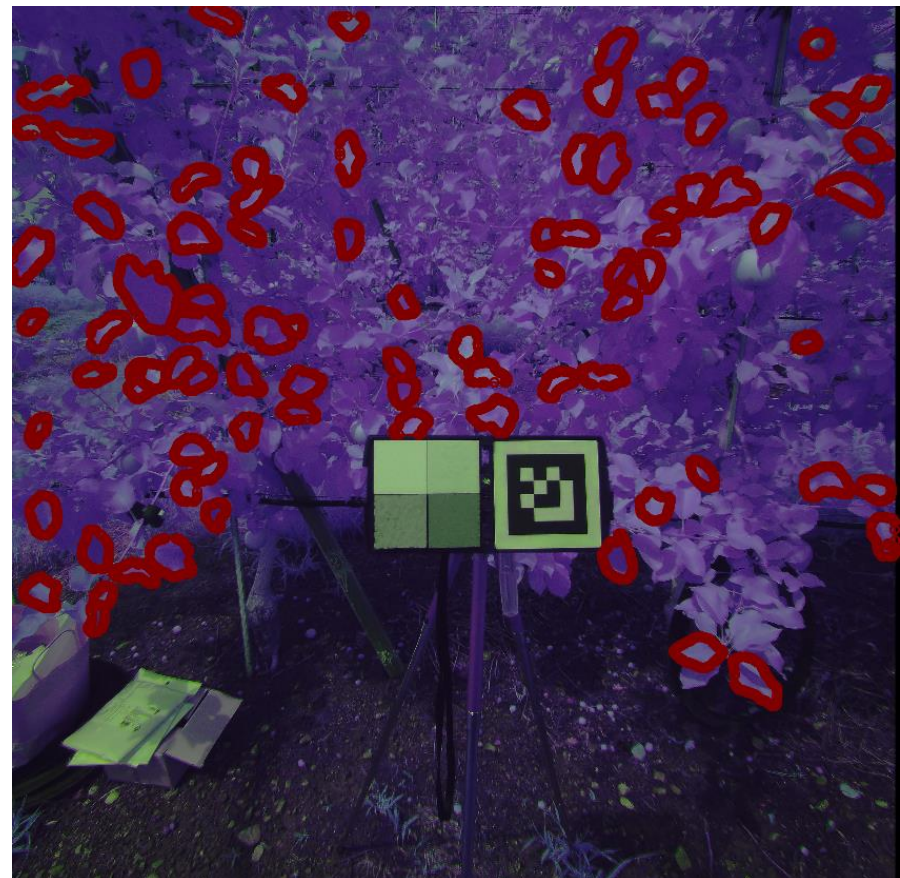
03 Instance Segmentation

데이터 준비 및 라벨링

Reverse Labeling – YOLO or SAM(Segment Anything Model)



YOLO

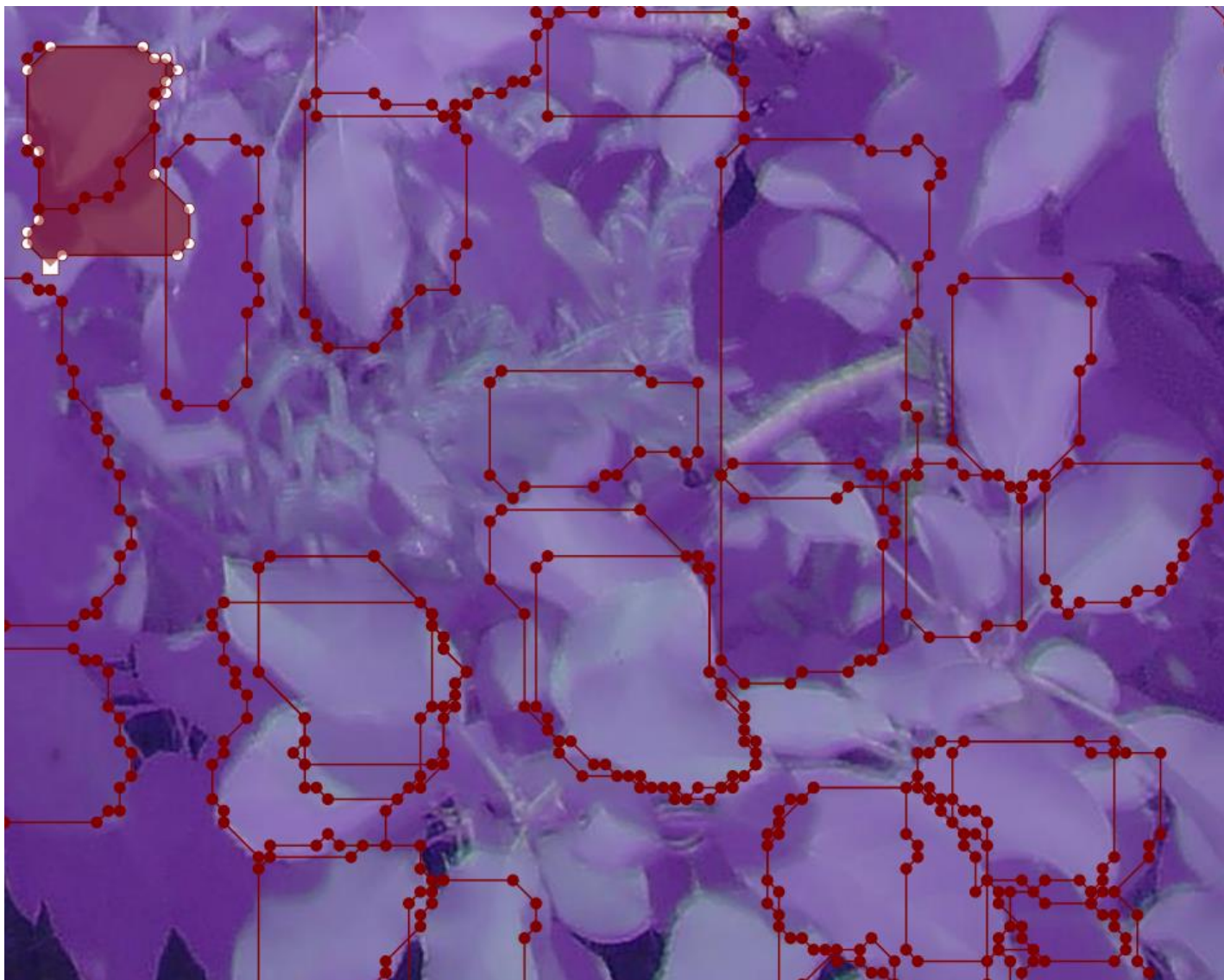


SAM

03 Instance Segmentation

데이터 준비 및 라벨링

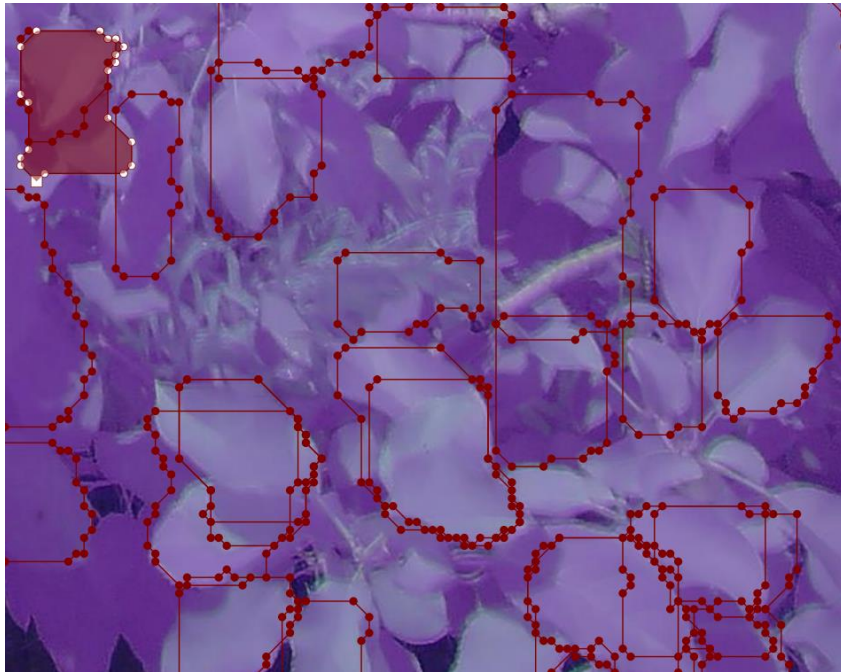
10장의 이미지, 20 epochs



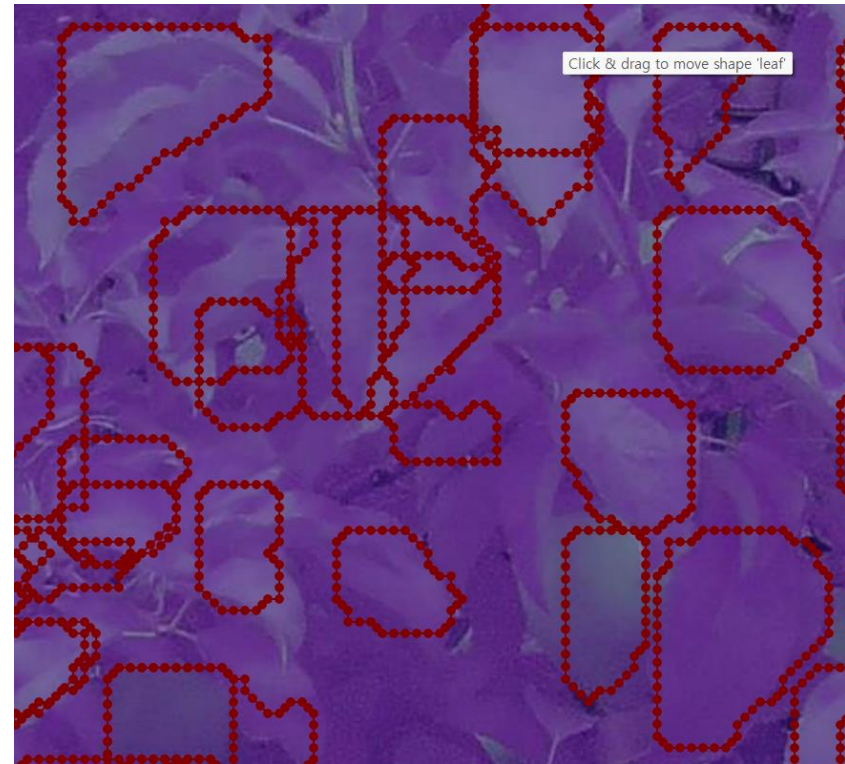
03 Instance Segmentation

모델학습 및 성능평가

Reverse Labeling



→
D(P-P) decrease



SAM을 이용하여 데이터셋의 증가

03 Instance Segmentation

모델학습 및 성능평가

YOLO8x-seg (80장의 RGN 이미지)

Video Prediction (Size : 1280)



문제 : Segmentation Instance가 적음, 잘못된 Segmentation
해결 : Channel spliting / NIR base 모델

03 Instance Segmentation

모델학습 및 성능평가

YOLO8x-seg (80장의 NIR 이미지, Sharpness Filter)

Video Prediction (Size : 1280)



03 Instance Segmentation

모델학습 및 성능평가

Video Size : 640



RGN



NIR

03 Instance Segmentation

모델학습 및 성능평가

GCI 계산 및 출력 - R/G/N 채널분할

```
import cv2
import os

def split_channels(image_path, output_dir):
    image = cv2.imread(image_path)
    if image is None:
        print(f"Failed to read image: {image_path}")
        return

    R, G, N = cv2.split(image)

    # Save each channel separately
    os.makedirs(output_dir, exist_ok=True)

    cv2.imwrite(os.path.join(output_dir, 'R_channel.png'), R)
    cv2.imwrite(os.path.join(output_dir, 'G_channel.png'), G)
    cv2.imwrite(os.path.join(output_dir, 'N_channel.png'), N)

    print(f"Channels saved to {output_dir}")

# Example usage
image_path = 'path_to_image.png'
output_dir = 'output_directory'
split_channels(image_path, output_dir)
```

03 Instance Segmentation

모델학습 및 성능평가

GCI 계산 및 출력 - GCI 계산



```
import numpy as np

def calculate_gci(R, N):
    # GCI = (NIR / Green) - 1
    GCI = (N.astype(float) / G.astype(float)) - 1
    return GCI

# Example usage
R = cv2.imread('path_to_R_channel.png', cv2.IMREAD_GRAYSCALE)
G = cv2.imread('path_to_G_channel.png', cv2.IMREAD_GRAYSCALE)
N = cv2.imread('path_to_N_channel.png', cv2.IMREAD_GRAYSCALE)

GCI = calculate_gci(R, N)
cv2.imwrite('output_directory/GCI.png', GCI * 255) # Scale to 0-255 for saving
```

$$GCI = \left(GCI = \frac{NIR}{Green + 1e - 7} - 1 \right)$$

소스코드

03 Instance Segmentation

모델학습 및 성능평가

GCI 계산 및 출력 – Sharpness 필터 적용



```
def apply_sharpness_filter(image):  
    kernel = np.array([[0, -1, 0],  
                       [-1, 5, -1],  
                       [0, -1, 0]])  
    sharp_image = cv2.filter2D(src=image, ddepth=-1, kernel=kernel)  
    return sharp_image
```

소스코드

03 Instance Segmentation

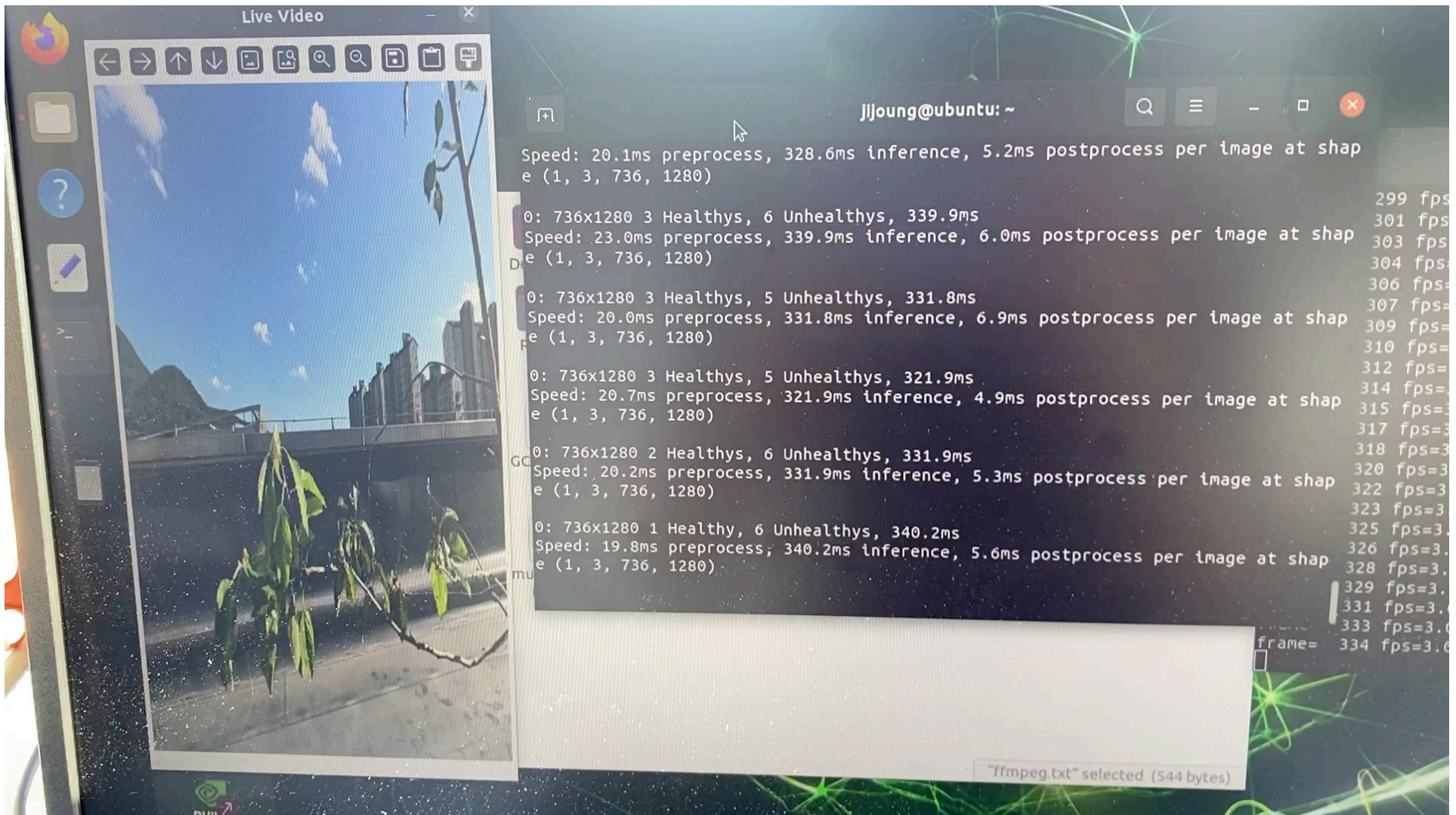
개선작업 및 최종평가

최종모델 (YOLOv8x-seg, Video Size : 640)



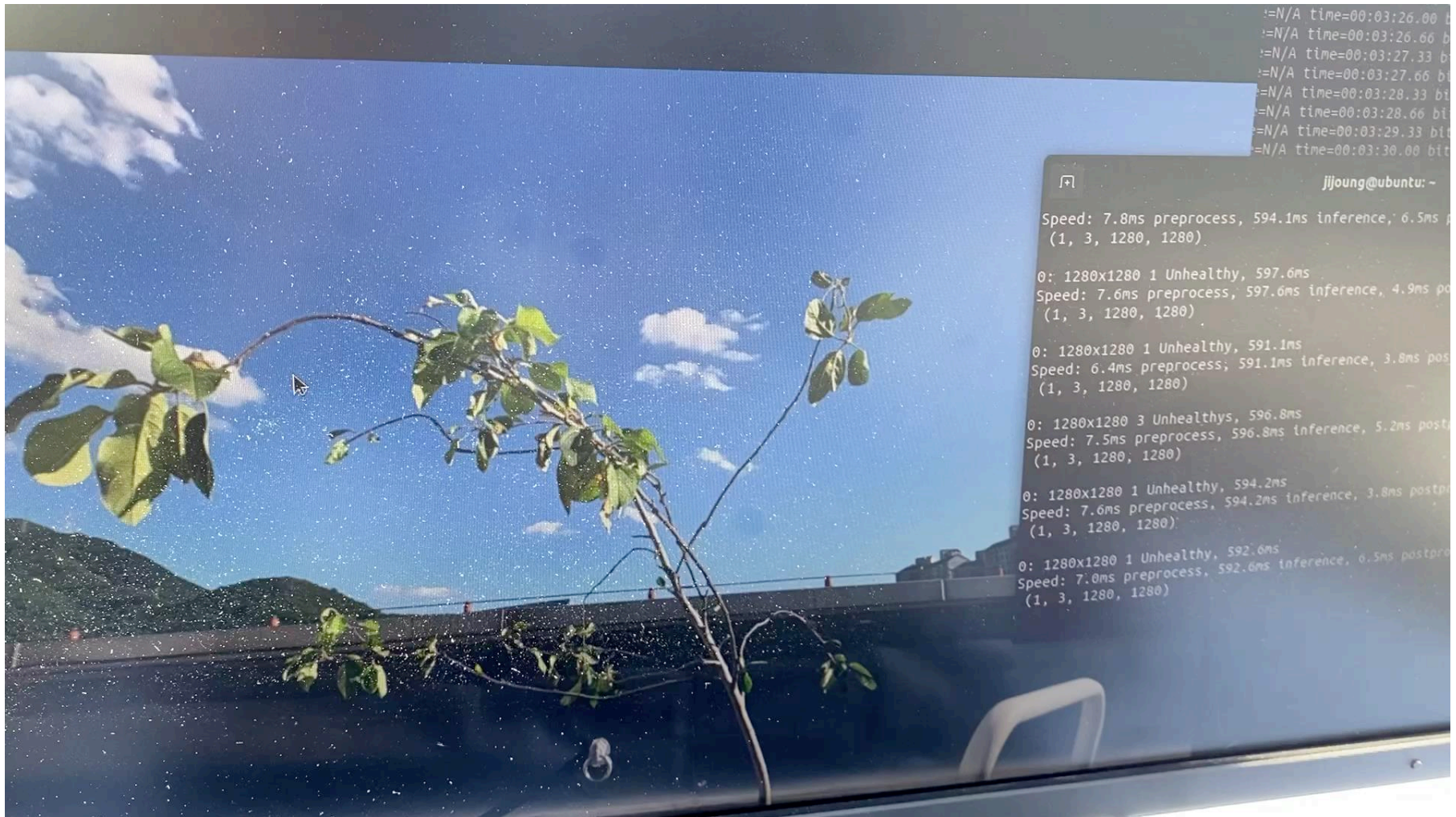
04 Real-time Streaming

Object Detection



04 Real-time Streaming

Object Detection



04 Real-time Streaming

Instance Segmentation

